Kuwait University
College of Life Sciences

Report III: System Design Specification

ISC 499 – Capstone Project
Spring 2025/2026

Instructor: Dr. Bader Alkhazi

Students: 2221191253 Rana Omar
2221168768 Mahnoor Akhtar

3-26-2026

Table of Contents

System Design **3**

1. Security Strategies 3

Authentication and Authorization 3

Session Management 3

Transport Layer Security 3

Rate Limiting 4

Input Validation and Injection Prevention 4

Data Privacy and Anonymized Logging 4

2. Integrity 4

Security-Based Integrity 4

Software Integrity 5

Data Integrity 5

3. Design Strategies 6

Design Approach 6

Design Paradigm 6

Design Benefits 6

User Interface Design **7**

Student Interface 8

1. Landing page 8

2. Chat Interface 9

Admin Interfaces 10

1. Log In Page 10

2. Dashboard 11

3. Content Management 12

4. Query Logs 13

Database Design **14**

Database Schema 14

Program Design **15**

Package Diagram 15

Frontend Package 15
Backend Package 15
Retrieval Augmented Generation (RAG) Package 16
Data Ingestion Package 16
Database Package 16
External Services 16
Design Class Diagram 17
Frontend 18
Backend 18
Retrieval Augmented Generation (RAG) Pipeline 18
Data Ingestion 18
Database 18
External Services 18
Physical Architecture Design 19
Architecture Diagram 19
Data Sources 20
Processing 20
Database 20
Backend 20
Frontend 20
Use Case Realization 20
1. Manage Chatbot Knowledge Base 21
2. Submit Query 23
3. Estimate GPA 25

System Design

1. Security Strategies

The Academic Chatbot system is designed as an open-access platform, meaning no authentication is required from general student users. However, since the system handles academic information, processes user queries, and includes a privileged administrative control panel, a layered security approach has been adopted across all components of the system. The following security measures define how the system protects data, controls access, and maintains integrity throughout its operation:

Authentication and Authorization

Access to the administrative control panel is restricted to authorized Kuwait University staff through Supabase Auth. Upon successful login, Supabase issues a JSON Web Token (JWT) that is stored client-side and included in subsequent requests to the Flask backend. The backend verifies the token on each request using Supabase's JWT secret, ensuring that only authenticated administrators can access functions such as knowledge base management and query log review. Tokens are configured with a defined expiration period, after which the admin must re-authenticate. Unauthorized users attempting to access the admin panel are denied access and presented with an authentication error.

Session Management

Administrative sessions are time-limited and expire after a defined period of inactivity, reducing the risk of unauthorized access through unattended sessions.

Transport Layer Security

All communication between the client and the server is conducted exclusively over HTTPS, ensuring that data transmitted between the user's device and the system is encrypted in transit. This applies to both the general user-facing chat interface and the administrative control panel, protecting query content, system responses and administrative actions from interception or tampering.

Rate Limiting

To protect the system from excessive or abusive usage, rate limiting is applied to the public chat endpoint using Flask-Limiter. This restricts the number of queries a single client can submit within a defined time window, preventing resource exhaustion and controlling costs associated with external API calls to OpenAI.

Input Validation and Injection Prevention

All user inputs submitted through the chat interface and the administrative panel are validated and sanitized on the server side before processing. This prevents common injection attacks, including SQL injection and prompt injection.

- SQL injection is mitigated through Supabase's client library, which uses parameterized queries by default, ensuring that user input is never directly embedded into database queries.
- Prompt injection is mitigated by maintaining a clear separation between the system prompt and user input within the OpenAI API request structure, and by enforcing a character limit on user queries to prevent attempts to override the system's instructions.

Data Privacy and Anonymized Logging

Since the user-facing interface operates without user authentication, no personally identifiable information is collected, stored, or linked to any interaction. Query logs maintained by the system record only the submitted query text, the generated response, and a timestamp. Users are advised not to include personal identifiers such as their name or student ID in their queries. GPA inputs entered during the estimation process are handled entirely within the session and do not persist to the database.

2. Integrity

Integrity ensures that the system's software and data remain accurate, consistent, and trustworthy throughout operation. The following subsections describe how integrity is maintained at each level:

Security-Based Integrity

The security measures described in Section 1 play a direct role in maintaining system integrity. Authentication and authorization ensure that only authorized administrators can modify

the knowledge base, preventing unauthorized changes to system content. Transport layer security ensures that data transmitted between the user interface and backend services cannot be altered during transit. Input validation and sanitization prevent malicious inputs from corrupting stored data.

Software Integrity

Software integrity focuses on ensuring the system's codebase remains reliable, secure, and free from unauthorized modifications. In developing the system, GitHub will be used as a version control system, which allows all code changes to be tracked, documented, and reviewed before being merged into the main codebase. This approach provides a complete history of modifications and enables reverting to previous versions if issues arise. Before any update is deployed to production, the code undergoes testing to identify potential bugs or vulnerabilities. The system will also follow a change management process where proposed modifications are reviewed, tested, and approved before implementation, preventing unplanned changes from affecting system stability.

Data Integrity

Data integrity ensures that information stored and processed by the system remains accurate, consistent, and trustworthy. When users submit queries, the system validates the input before processing to ensure that empty, excessively long, or malformed queries are rejected. This prevents invalid data from entering the system and affecting its performance.

The chatbot knowledge base is sourced exclusively from official Kuwait University channels, including the KU website, KU portal, and verified social media accounts, ensuring that all information provided to students is accurate and institutionally approved. Each document added to the knowledge base is timestamped to track when it was uploaded or last updated, allowing content to remain current and traceable to its origin. Only authenticated administrators can modify the knowledge base, while students interact with the system through a read-only interface that provides information without allowing changes. All updates performed through the administrative panel are validated before being stored in the database, and administrative actions are logged to support traceability and accountability.

3. Design Strategies

Designing a system that is easy to build, maintain, and scale requires deliberate and thoughtful decisions from the start. The following strategies outline the approach taken to ensure the KU AI Chatbot remains structured, manageable, and adaptable throughout its development lifecycle.

Design Approach

The system follows a top-down design approach, starting with a broad view of what the system needs to do and working down into the details. Rather than jumping straight into individual components, the chatbot was first defined as a complete system, one that answers student queries, and then broken down into subsystems: data collection, processing, storage, backend logic, and frontend interface.

Design Paradigm

A functional-oriented design paradigm was chosen to keep things focused and straightforward. Each part of the system is responsible for one specific task, whether it is scraping websites or social media content, extracting content from PDFs, managing document chunking and retrieval, or handling API requests. Keeping these functions independent from one another makes it much easier to test them in isolation and adjust them later without causing issues in other system functionalities.

Design Benefits

1. **Modularity:** meaning that the system is divided into self-contained components, each with a clear responsibility. They communicate through well-defined interfaces, so one component can be updated, debugged, or replaced without touching the others. This makes troubleshooting simpler and leaves room for future additions, like new data sources or a different language model.
2. **Reusability:** which cuts down on duplicated effort and keeps behavior consistent across the system. The document processing pipeline works the same way regardless of where the content comes from, and the same logic handles embedding generation and search whether the source is a website, a PDF, or a social media post. This consistency reduces redundancy and makes the system easier to maintain over time.

User Interface Design

The User Interface Design for the system is divided into two sections: the student-facing chat interface and the administrative control panel. The system is responsive and adapts across desktop, tablet, and mobile devices. The interface also adheres to accessibility standards established in the system requirements. Text contrast ratios meet readability guidelines to support users with visual impairments. The voice-to-text input feature accommodates users who have difficulty typing.

The student chat follows a language-mirroring policy, the chatbot detects the student's input language and responds in the same language. In addition, the Admin interfaces can be switched between Arabic and English. Student interface screenshots in this section are presented in mobile view while the admin panel screenshots are presented in desktop view to reflect realistic usage.

Student Interface

The student interface consists of two main screens as described below. They are designed to provide a simple and welcoming experience without requiring any login or registration.

1. Landing page

The landing page is the entry point for students when using the chatbot. It provides an animated entrance with the bot logo blinking and growing into view, creating a welcoming first impression and a friendly user experience for students using the chatbot.

Figure 1: Chatbot Landing Interface

2. Chat Interface

The chat interface is the core screen where students interact with the chatbot, following a familiar messaging layout. It includes an input bar where students can type their inquiries or use the voice-to-text feature instead. The inquiries are then displayed in the conversation area along with the chatbot responses, which include attached source links so students can verify the information.

Figure 2: Student Chat Interface

Admin Interfaces

The administrative panel consists of four main interfaces. After logging in, the admin is directed to the Dashboard, which serves as the central hub. From there, the admin can navigate to the Content Management page to manage the knowledge base or the Query Logs page to review student interactions.

1. Log In Page

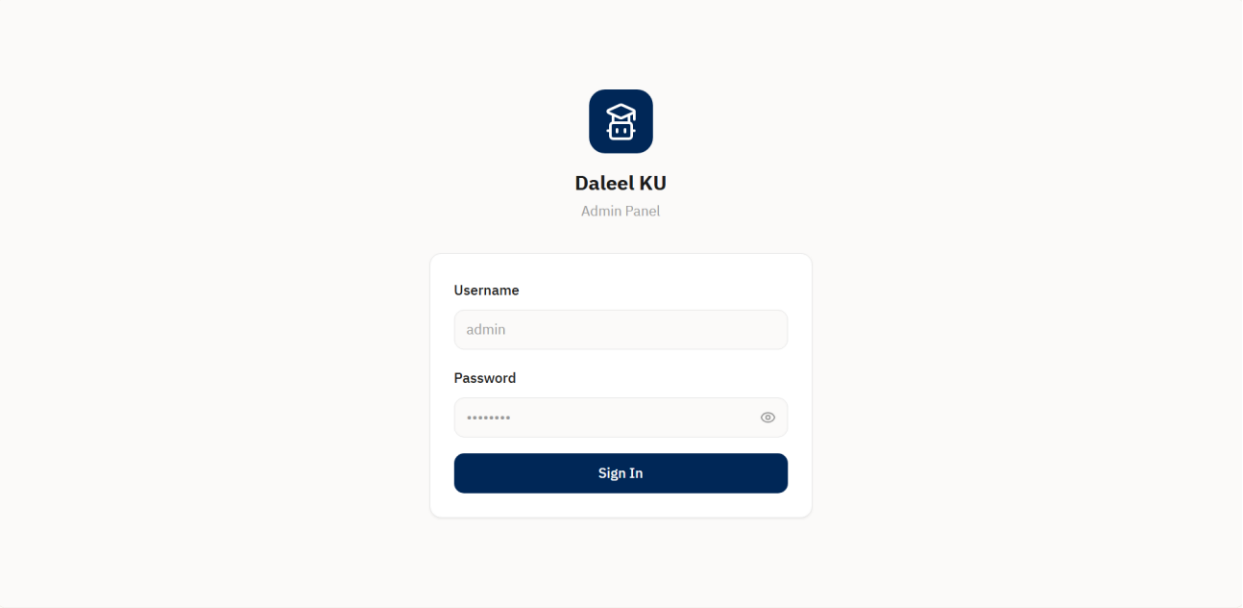

Figure 3: Admin Login Page

The Admin interface begins with an Admin Login page providing secure access to the system's administrative panel. Authorized Kuwait University staff enter their username and password then click Sign In to authenticate making access exclusively restricted to authenticated administrators, with invalid credentials triggering an error message.

2. Dashboard

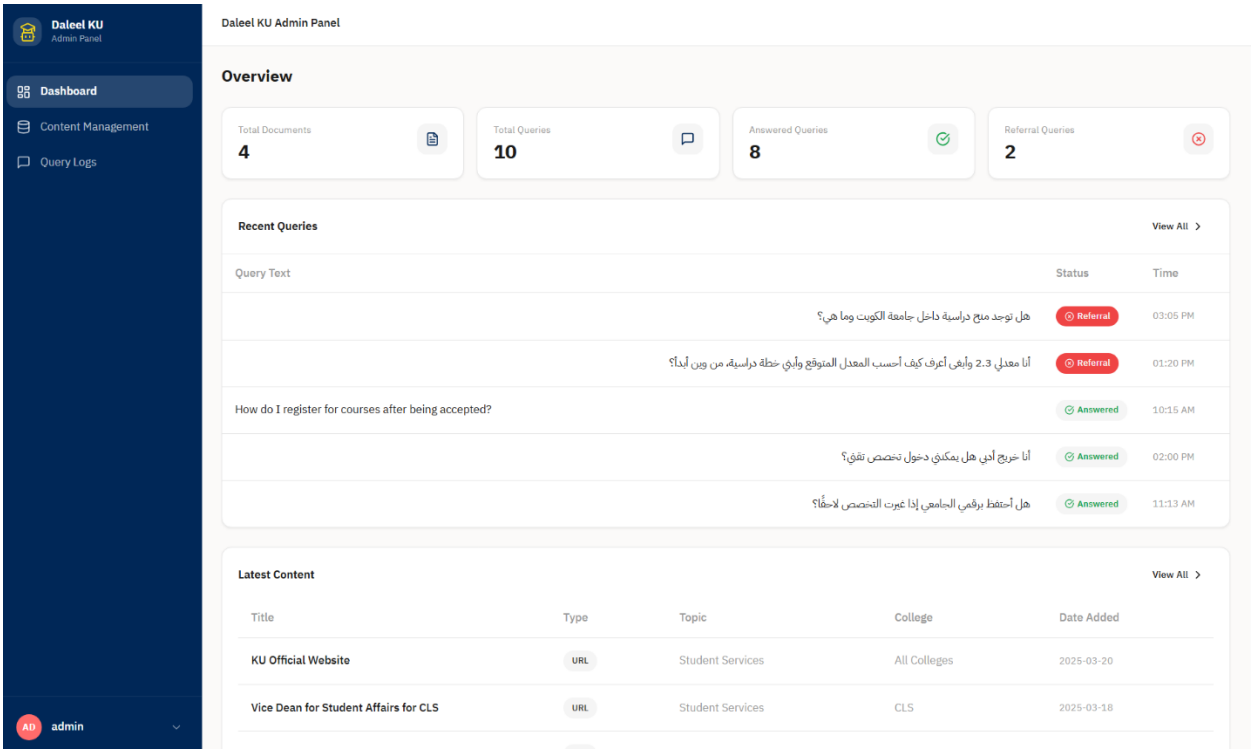

Figure 4: Admin Dashboard Interface

The Dashboard is the admin's main overview page, structured into three sections:

Overview: Four visual cards at the top display real-time system statistics to give the admin an instant snapshot of system performance: Total Documents in the knowledge base, Total Queries received, Answered Queries, and Referral Queries. Answered queries are those where the chatbot successfully returned a response from the knowledge base. Referral queries are those where no relevant information was found and the student was redirected to the appropriate Kuwait University department.

Recent Queries: A table showing the five most recent student queries, each displaying the query text, status (Answered or Referral), and time. A View All button navigates to the full Query Logs page.

Latest Content: It displays the most recently added documents, showing each document's title, type, topic, college, and date added. A View All button navigates to the full Content Management page.

3. Content Management

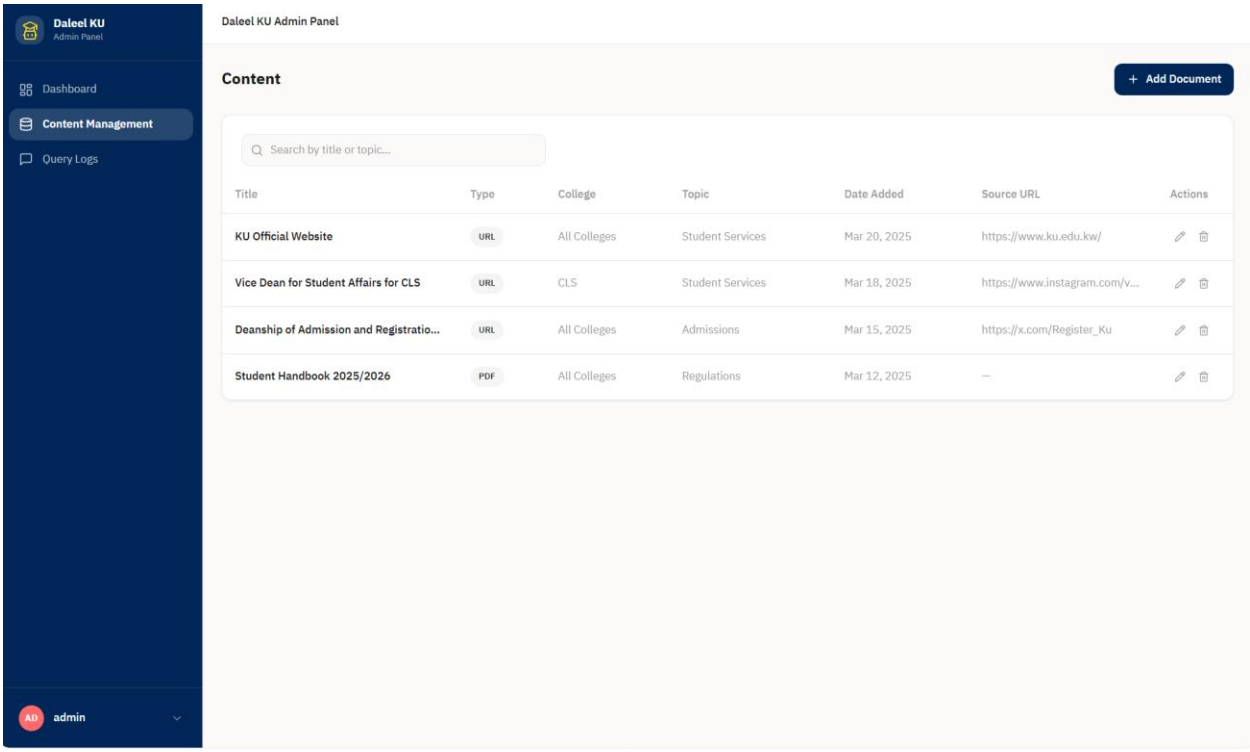

Figure 5: Admin Content Management Interface

The Content Management page allows administrators to manage the chatbot's content, structured as follows:

Content Table: This displays all documents in the knowledge base with the following columns: Title, Type (PDF, URL, etc.), College, Topic, Date Added, and Source URL.

Actions: Each row has an edit icon to modify a document's details and a delete icon to remove it from the knowledge base.

Add Document: Add Document button in the top right allows administrators to add new content to the knowledge base and classify it by document type, source URL, topic, and college.

Search: A search bar is also integrated to allow administrators to filter documents by title or topic.

4. Query Logs

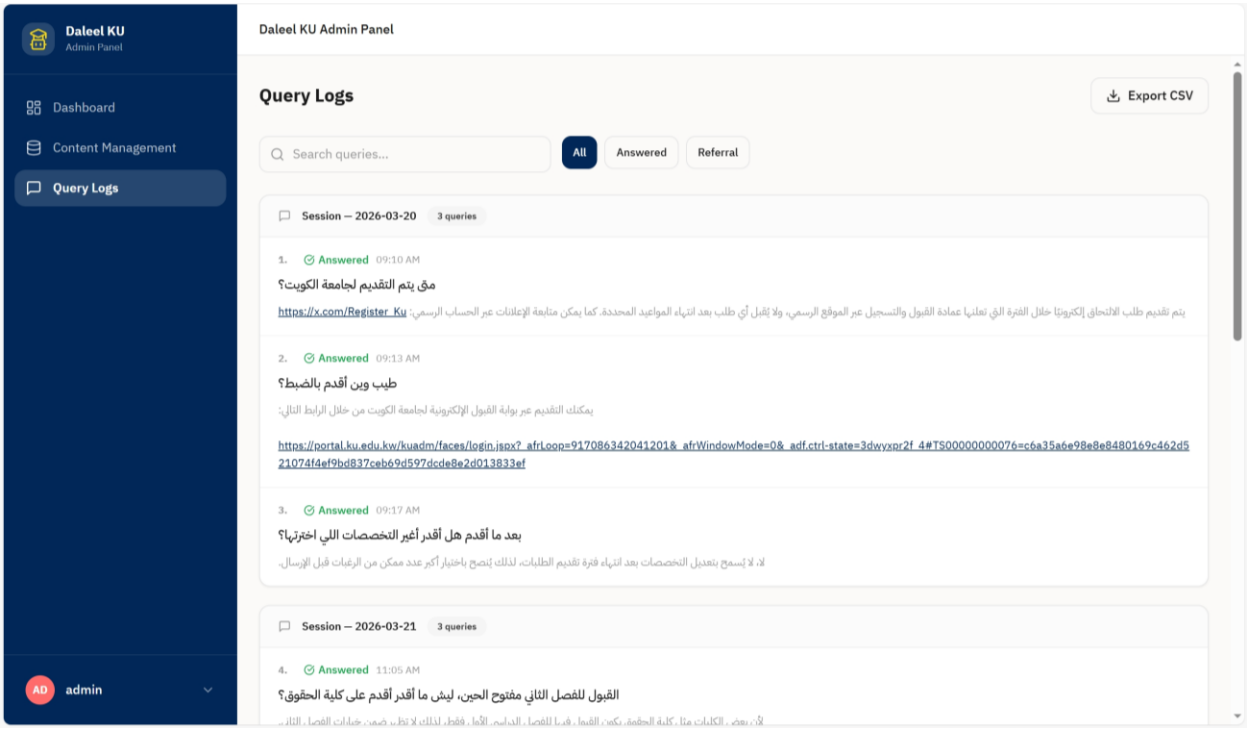

Figure 6: Admin Query Logs Interface

The Query Logs page allows administrators to monitor all student interactions with the chatbot, structured as follows:

Filters: A search bar and three filter buttons (All, Answered, Referral) allow administrators to quickly find and narrow down specific queries.

Session-Based Logs: Queries are grouped by session, with each session showing its date and total query count. Within each session, individual queries are displayed with their full query text, the chatbot's response, status (Answered or Referral), and timestamp.

Export CSV: Export CSV button on the top right allows administrators to download the full query log for reporting or analysis purposes.

Database Design

Database Schema

The database schema defines how data is organized and stored within the system. It was derived from the Entity Relationship Diagram (ERD) developed in Report II, with further enhancements based on the system design requirements identified during this phase. It consists of eight tables designed to support the chatbot's core functionality, including storing user queries, managing administrator accounts, and organizing the knowledge base content.

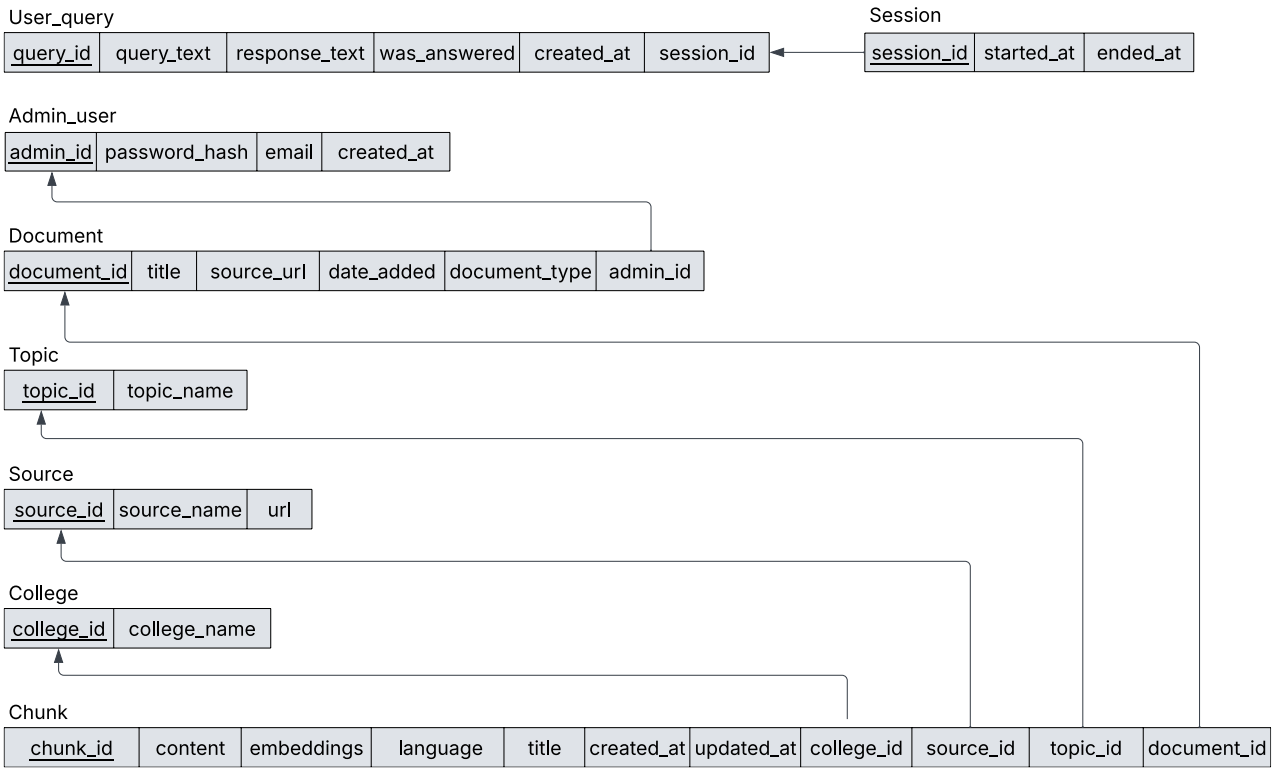

Figure 7: Database Schema Diagram

Program Design

Package Diagram

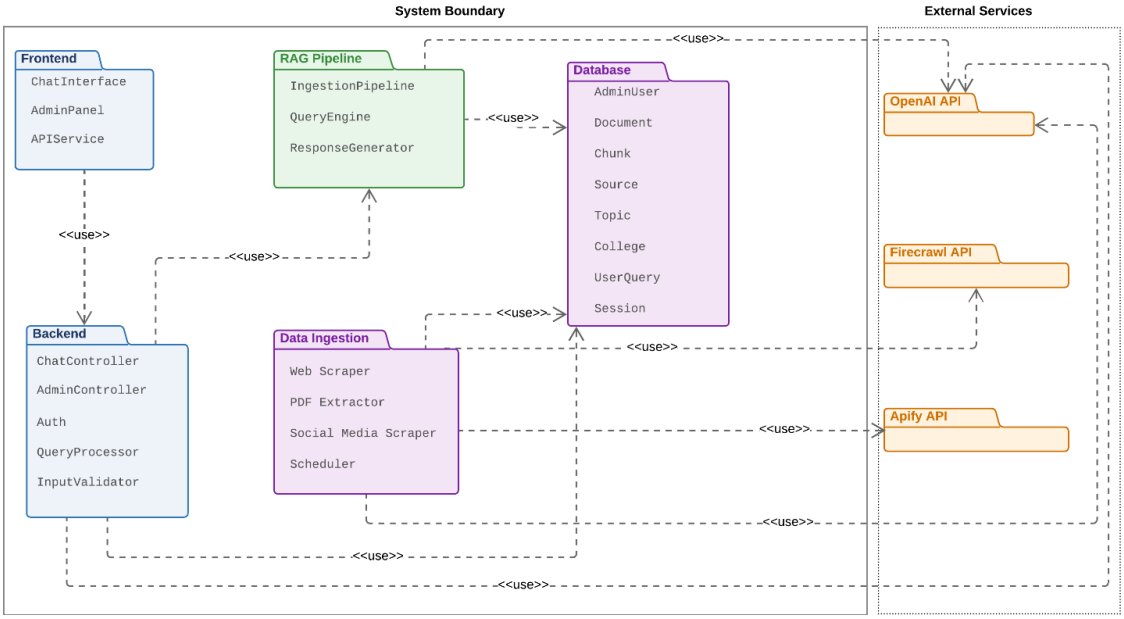

Figure 8: System Package Diagram

The package diagram shows the logical organization of the system across two boundaries: the System Boundary containing all internally developed packages, and the External Services boundary containing third-party APIs. All dependencies are represented as <<use>> relationships indicated by dashed arrows, as all inter-package communication occurs through defined interfaces and APIs.

Frontend Package

Frontend package represents the user-facing layer of the system, containing the ChatInterface and AdminPanel components through which students and administrators interact with the system, and the APIService class responsible for all communication with the backend.

The Frontend depends on the Backend to process all submitted queries and administrative requests.

Backend Package

Backend package serves as the core of the system, containing the ChatController and AdminController which handle incoming requests, the Auth module which manages

administrative authentication and session management, the QueryProcessor which identifies and routes query types, and the InputValidator which validates submitted data before processing.

The Backend depends on the RAG Pipeline to process general academic queries, on the Database for data storage and retrieval, and on the OpenAI API directly for tasks such as GPA estimation.

Retrieval Augmented Generation (RAG) Package

RAG Pipeline package contains the retrieval-augmented generation logic of the system, containing the IngestionPipeline which handles document loading, chunking, and embedding generation, the QueryEngine which performs semantic vector search and context building and the ResponseGenerator which constructs the final prompt and generates the chatbot's response.

The RAG Pipeline depends on the Database to retrieve relevant document chunks through vector search, and on the OpenAI API to generate embeddings during ingestion and synthesize responses during retrieval.

Data Ingestion Package

Data Ingestion package contains the WebScraper, PDFExtractor, and SocialMediaScraper which collect content from KU's official websites, PDF documents, and official social media accounts and the Scheduler which automates all ingestion tasks on a defined schedule.

The package depends on the Database to store all processed and chunked content, on the Firecrawl API for automated website crawling, on the Apify API for automated social media scraping, and on the OpenAI API's vision capability to extract textual content from images.

Database Package

Database package represents the system's data layer, containing all eight entity classes defined in the Database Schema previously: AdminUser, Document, Chunk, Source, Topic, College, Session, UserQuery.

External Services

External Services boundary contains the OpenAI API, Firecrawl API, and Apify API, which are maintained outside the system boundary and accessed through <<use>> dependencies by the internal packages.

Design Class Diagram

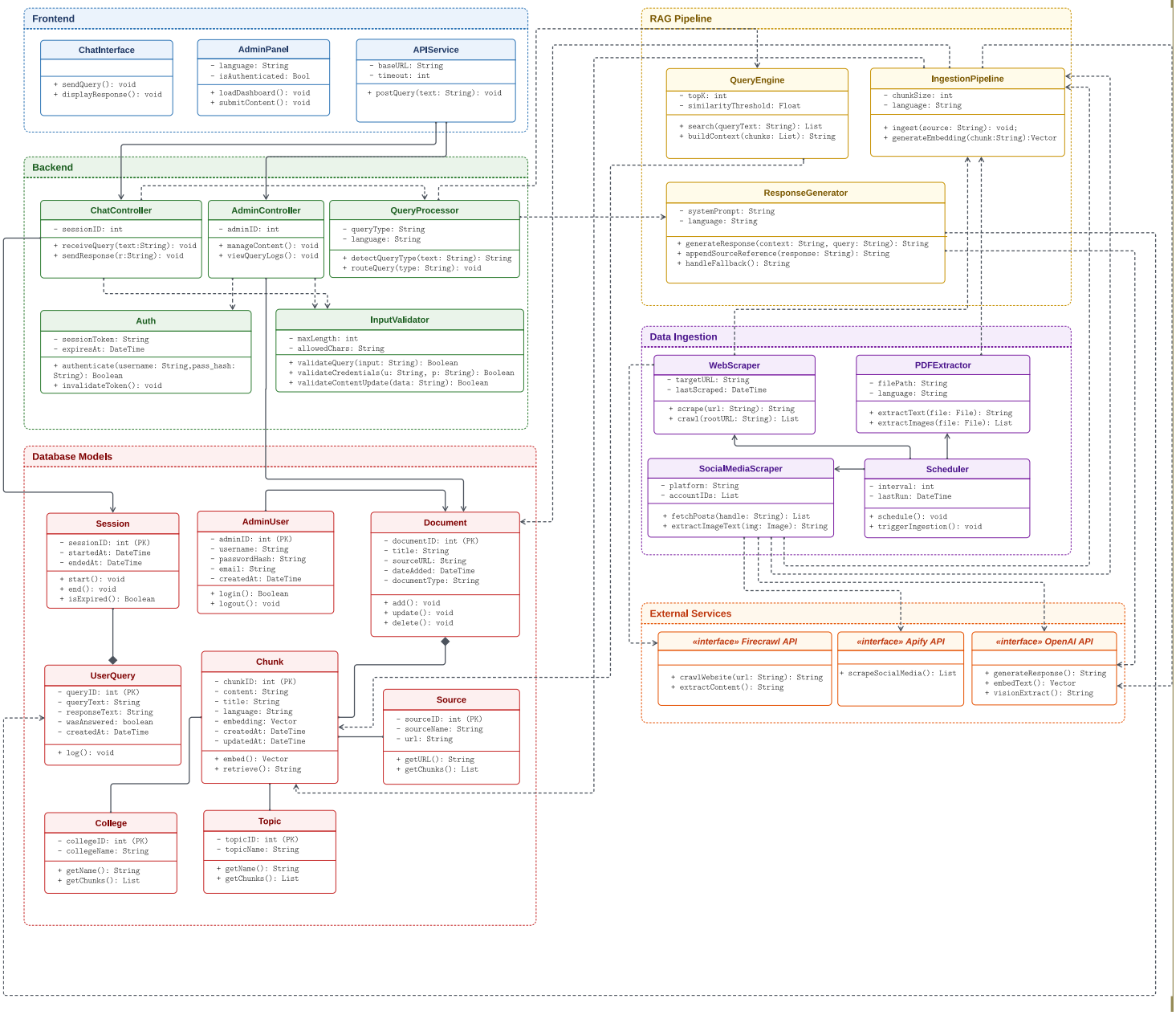

Figure 9: Chatbot Design Class Diagram

The Design Class Diagram above shows the structure of the system, organized into six sections: Frontend, Backend, RAG Pipeline, Data Ingestion, Database Models and External Services. Different relationships are also represented in the diagram, including association, navigable association, composition, and dependency, capturing the structural connections and usage dependencies between classes across all sections. Below are the system's design classes and their relationships, organized by section as follows:

Frontend

This contains the ChatInterface and AdminPanel as the student and admin-facing components respectively and APIService which handles all communication between the frontend and the backend.

Backend

This consists of the ChatController and AdminController which handle incoming requests, Auth which manages admin authentication and session tokens, QueryProcessor which detects query type and language and routes accordingly, and InputValidator which validates all inputs before processing.

Retrieval Augmented Generation (RAG) Pipeline

This section contains IngestionPipeline which loads, chunks, and embeds documents into the knowledge base, QueryEngine which performs semantic vector search and builds context from retrieved chunks, and ResponseGenerator which constructs the prompt, generates the response via OpenAI, handles fallback messages, and appends source references.

Data Ingestion

This section contains WebScraper, PDFExtractor, and SocialMediaScraper which collect content from KU's website, PDF documents, and official social media accounts respectively, and Scheduler which automates all ingestion tasks on a defined interval.

Database

This section contains all eight entity classes: AdminUser, Session, Document, Chunk, Source, Topic, College, and UserQuery showing the system's data model as defined in the database schema.

External Services

This section consists of three interface classes, OpenAI API, Firecrawl API, and Apify API representing the third-party services the system depends on for response generation, website crawling and social media scraping respectively.

Physical Architecture Design

Architecture Diagram

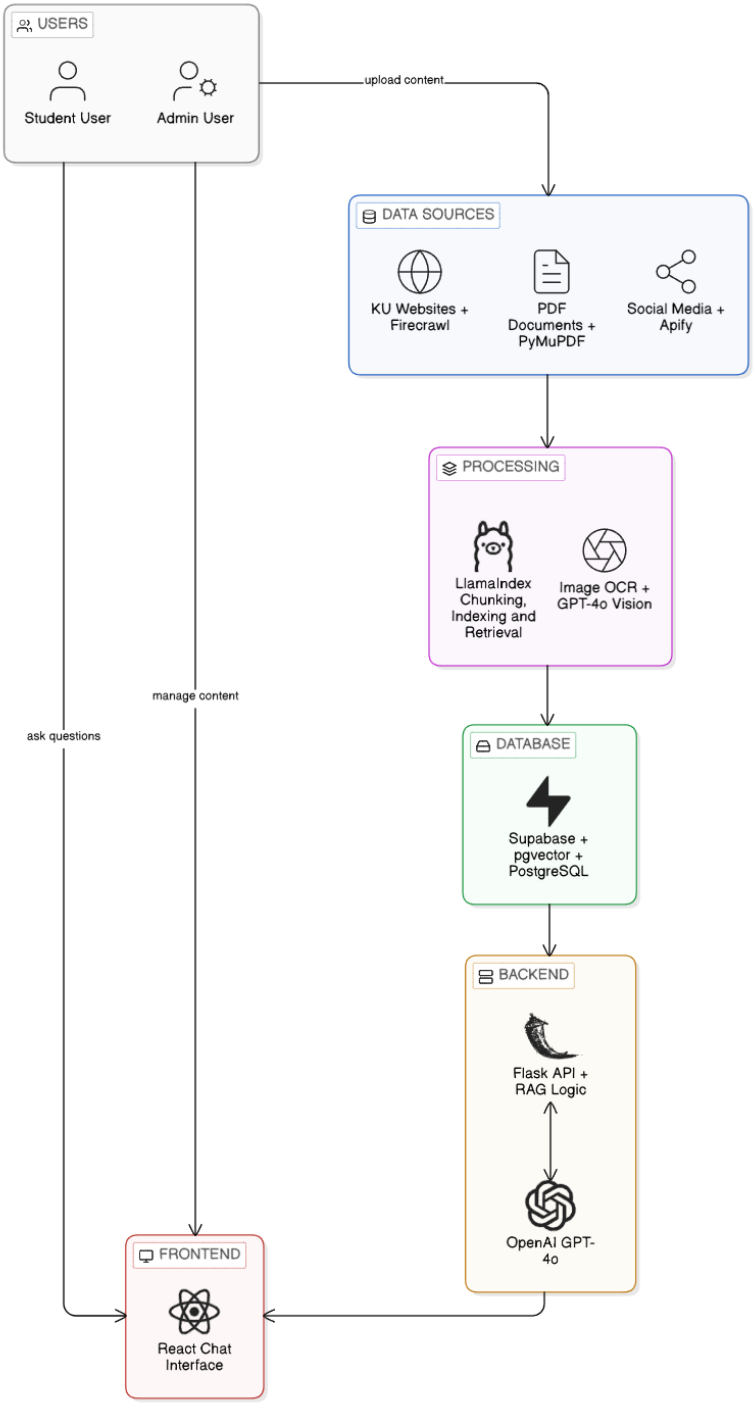

Figure 10: Physical Architecture Design

The physical architecture design diagram of the system presented above was designed to meet the functional and non-functional requirements identified in Report II. The following describes the purpose of each layer:

Data Sources

The system collects information from three main sources: KU websites and social media content, scraped using Firecrawl and Apify respectively, and PDF documents using PyMuPDF for content extraction.

Processing

To process the data from the knowledge base, LlamaIndex transforms the raw content into searchable chunks for the RAG system and generates vector embeddings that enable semantic search across the knowledge base. For image-based content, GPT-4o Vision extracts text through OCR before the content is processed and stored.

Database

Supabase serves as the central data store, providing PostgreSQL for relational database and pgvector for storing and querying vector embeddings.

Backend

For the backend, Flask handles incoming API requests from the frontend, manages RAG queries through LlamaIndex, and communicates with OpenAI GPT-4o for response generation and GPA estimation.

Frontend

The React interface provides students with a chat experience and allows administrators to manage content through a dedicated dashboard.

Use Case Realization

Use case realization describes how system use cases are carried out through interactions between components. Sequence diagrams were selected as the interaction diagram type for this system, as they effectively capture the order of messages exchanged between participants over time. Three main use cases labelled UC01, UC02, UC05 in Report 2 were chosen for realization based on their centrality to the system:

1. Manage Chatbot Knowledge Base

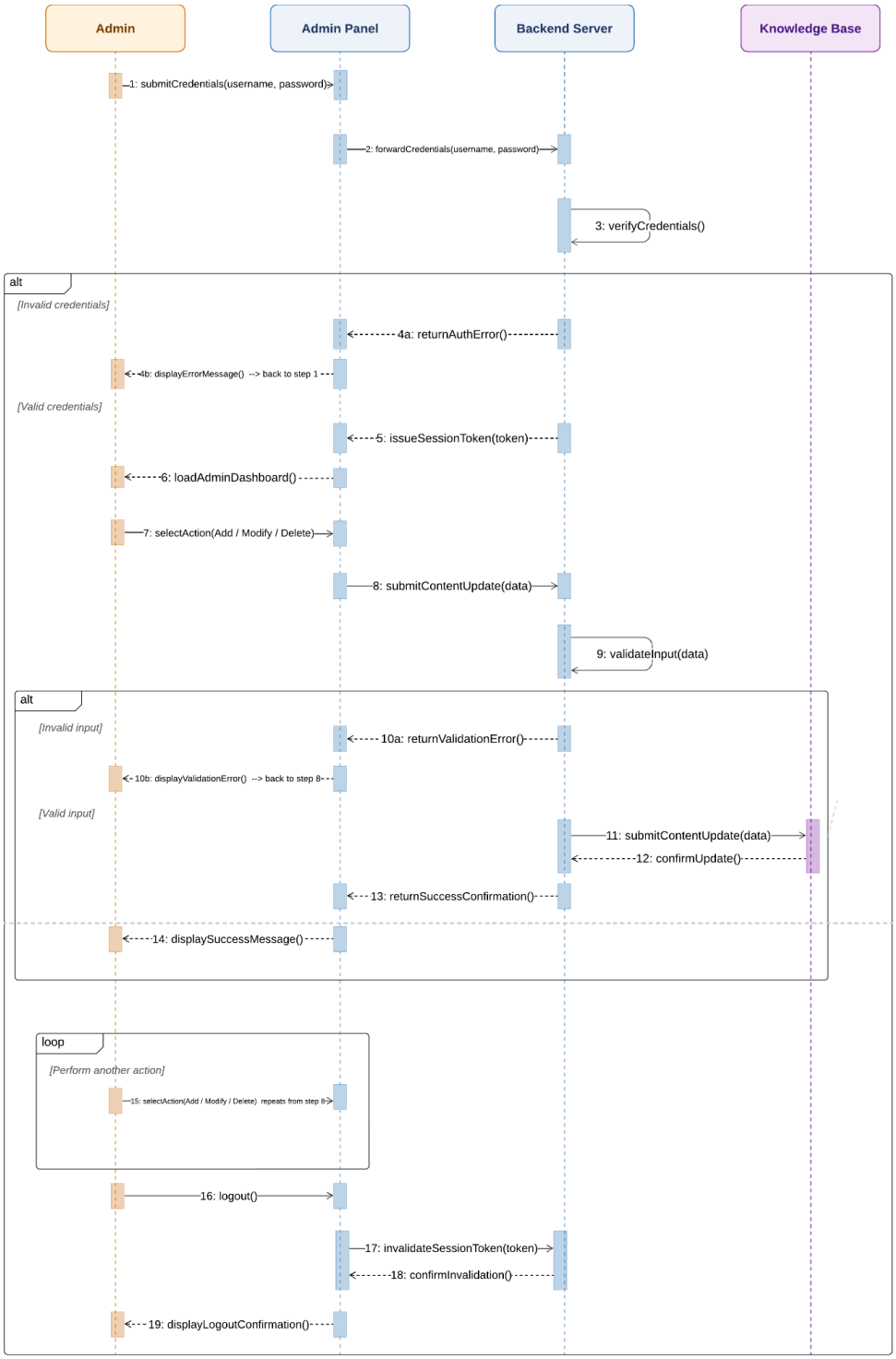

Figure 11: Sequence Diagram - UC01 Manage Knowledge Base

This diagram shows the interaction flow between system components when an authorized administrator manages the chatbot knowledge base. The flow proceeds as follows:

- **Authentication:** The admin submits their credentials through the Admin Panel, which forwards them to the Backend Server. The Backend Server internally verifies the credentials.
- **Alt Fragment - Credential Validation:** Two outcomes are possible. If credentials are invalid, an authentication error is returned and displayed to the admin, who is prompted to retry. If credentials are valid, a session token is issued, and the Admin Dashboard is loaded.
- **Action Selection and Content Validation:** The admin selects a content management action (Add, Modify, or Delete) through the Admin Panel, which forwards the content update to the Backend Server. The Backend Server internally validates the submitted content before proceeding.
- **Alt Fragment - Input Validation:** Two outcomes are possible. If the input is invalid, a validation error is returned and displayed to the admin, who is prompted to correct and resubmit. If the input is valid, the Backend Server writes the content to the Knowledge Base, which confirms the update. A successful confirmation is then returned and displayed to the admin.
- **Loop Fragment:** If the admin chooses to perform another action, the flow repeats from the action selection step.
- **Logout:** The admin initiates logout, upon which the Admin Panel sends a session token invalidation request to the Backend Server. The Backend Server confirms the invalidation, and a logout confirmation is displayed, ending the session.

2. Submit Query

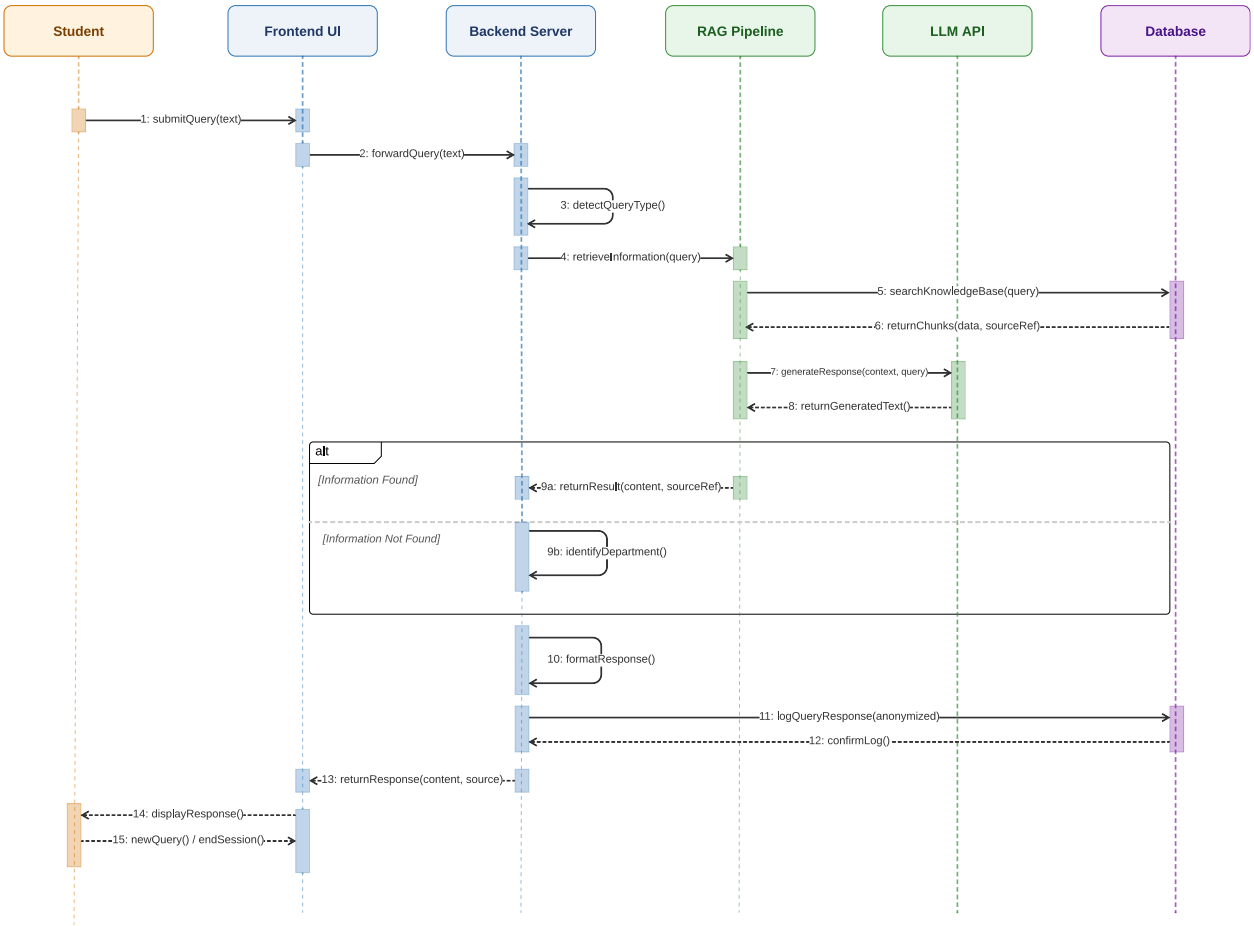

Figure 12: Sequence Diagram - UC02 Submit Query

The diagram shows the interaction flow between system components when a student submits a query. The flow proceeds as follows:

- **Query Submission:** The student submits a query through the Frontend UI, which forwards it to the Backend Server.
- **Query Type Detection:** The Backend Server identifies whether the query is a general academic question or a GPA-related request and routes it accordingly.
- **Knowledge Retrieval:** For general queries, the Backend Server delegates to the RAG Pipeline, which searches the Knowledge Base and retrieves relevant text chunks along with their source references.

- **Response Generation:** The retrieved content is passed to the LLM API, which generates a context-aware natural language response.
- **Alternative Fragment:** Two outcomes are possible at this stage. If relevant information is found, the result and source reference are returned and formatted into a response. If no information is found, the Backend Server identifies the appropriate official department to redirect the student to.
- **Logging:** Regardless of outcome, the query and response are anonymously logged to the Database for future knowledge base improvement.
- **Response Display:** The formatted response is returned to the Frontend UI and displayed to the student, who may then submit a new query or end the session.

3. Estimate GPA

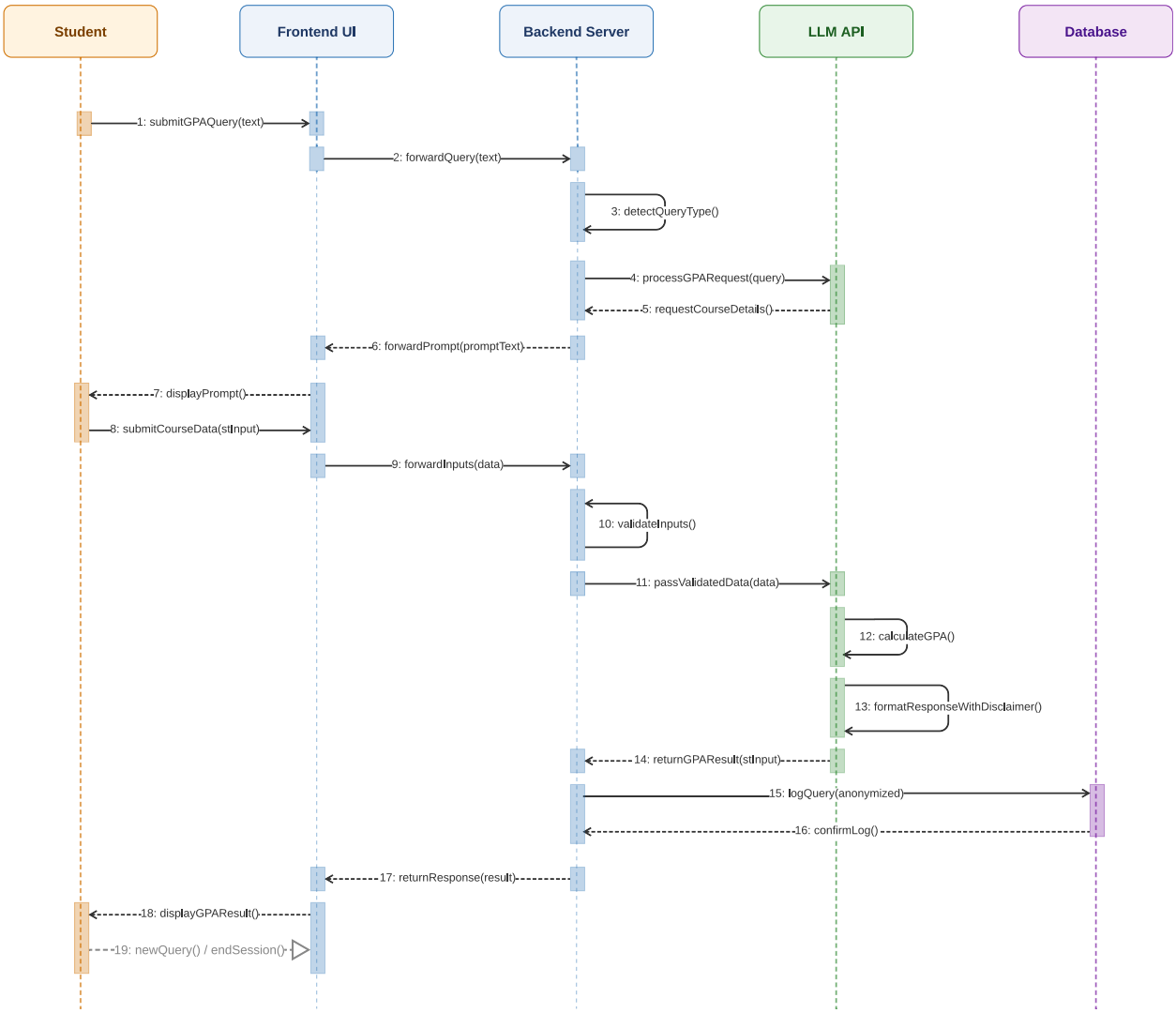

Figure 13: Sequence Diagram - UC05 Estimate GPA

The diagram shows the interaction flow between system components when a student requests a GPA estimation. The flow proceeds as follows:

- **Query Submission**: The student submits a GPA-related query through the Frontend UI, which forwards it to the Backend Server.
- **Query Type Detection**: The Backend Server identifies the query as a GPA estimation request and routes it to the LLM API for processing.

- **Course Details Request**: The LLM API generates a prompt requesting the student's course information, including grades, credit hours, previous cumulative GPA, previous registered units, and any retaken courses. This prompt is forwarded back through the Backend Server to the Frontend UI and displayed to the student.
- **Data Submission**: The student provides the requested course data, which is forwarded through the Frontend UI to the Backend Server.
- **Input Validation**: The Backend Server validates the submitted inputs to ensure grades are within the accepted scale, credit hours are positive integers, and optional fields such as previous GPA fall within the valid range.
- **GPA Calculation**: Once validated, the data is passed to the LLM API, which calculates both the semester GPA and the expected cumulative GPA using the Kuwait University grade scale defined in its system prompt.
- **Response Formatting**: The LLM API formats the result into a user-friendly response and appends a disclaimer stating that the calculation is an estimate for reference only and should be verified against official academic records.
- **Logging**: The query and response are anonymously logged to the Database for analytics and system improvement purposes.
- **Response Display**: The formatted GPA result is returned to the Frontend UI and displayed to the student, who may then submit a new query or end the session.

Note: In the diagram, stInput represents the student input object containing expected grades for current courses and their credit hours, along with current GPA, and number of previously registered credits.